

Deep Reinforcement Learning for Solving Vehicle Routing Problems With Backhauls

Conghui Wang¹, Zhiguang Cao¹, Yaoxin Wu², Long Teng¹, *Member, IEEE*,
and Guohua Wu¹, *Senior Member, IEEE*

Abstract—The vehicle routing problem with backhauls (VRPBs) is a challenging problem commonly studied in computer science and operations research. Featured by linehaul (or delivery) and backhaul (or pickup) customers, the VRPB has broad applications in real-world logistics. In this article, we propose a neural heuristic based on deep reinforcement learning (DRL) to solve the *traditional* and *improved* VRPB variants, with an encoder–decoder structured policy network trained to sequentially construct the routes for vehicles. Specifically, we first describe the VRPB based on a graph and cast the solution construction as a Markov decision process (MDP). Then, to identify the relationship among the nodes (i.e., linehaul and backhaul customers, and the depot), we design a two-stage attention-based encoder, including a self-attention and a heterogeneous attention for each stage, which could yield more informative representations of the nodes so as to deliver high-quality solutions. The evaluation on the two VRPB variants reveals that, our neural heuristic performs favorably against both the conventional and neural heuristic baselines on randomly generated instances and benchmark instances. Moreover, the trained policy network exhibits a desirable capability of generalization to various problem sizes and distributions.

Index Terms—Deep reinforcement learning (DRL), logistics, neural heuristic, two-stage attention, vehicle routing problem (VRP).

NOMENCLATURE

Sets

- $\mathcal{G}$ Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is set of nodes and $\mathcal{E}$ is arc set.
- $\mathcal{L}$ Set of linehaul nodes (delivery customers), $\mathcal{L} = \{1, 2, \dots, m\}$.

Manuscript received 18 March 2023; revised 12 November 2023 and 8 January 2024; accepted 23 February 2024. Date of publication 29 March 2024; date of current version 1 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62373380 and Grant 62073341, and in part by the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 Grant. (*Corresponding author: Guohua Wu.*)

Conghui Wang and Guohua Wu are with the School of Traffic and Transportation Engineering, Central South University, Changsha 410075, China (e-mail: wangconghui@csu.edu.cn; guohuawu@csu.edu.cn).

Zhiguang Cao is with the School of Computing and Information Systems, Singapore Management University, Singapore 178902 (e-mail: zhiguangcao@outlook.com).

Yaoyin Wu is with the Department of Information Systems, Faculty of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, 5612 AZ Eindhoven, The Netherlands (e-mail: wyxacc@hotmail.com).

Long Teng is with the Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: eric-long.teng@polyu.edu.hk).

Data is available on-line at <https://github.com/wangconghui0/VRPB>
Digital Object Identifier 10.1109/TNNLS.2024.3371781

$\mathcal{B}$	Set of backhaul nodes (pickup customers), $\mathcal{B} = \{m + 1, \dots, n\}$.
$\mathcal{V}^*$	Set of all nodes excluding the depot node, $\mathcal{V}^* = (\mathcal{L}, \mathcal{B})$.
κ	Set of vehicles.
Parameters	
q_i	Backhaul or linehaul demand of node $i \in \mathcal{V}$.
c_{ij}^k	Travel cost of vehicle k ($k \in \kappa$) from i to j , $i, j \in \mathcal{V}$.
$\mathcal{D}$	Capacity of vehicle $k \in \kappa$.
Decision Variables	
x_{ij}^k	1 if vehicle k travels from node i to j 0 otherwise.
u_{ij}^k	Load of vehicle k when traveling on arc $(i, j) \in \mathcal{E}$, $k \in \kappa$.

I. INTRODUCTION

THE vehicle routing problem with backhauls (VRPBs), also known as the linehaul–backhaul problem, is an important yet challenging variant of the vehicle routing problem (VRP) [1]. In VRPB, vehicles are dispatched and routed to (alternatively) perform delivery and pickup services at different customer locations, with the goal to minimize certain costs by optimizing the routes and leveraging the vehicle capacity. In comparison with other VRP variants, the VRPB is proposed based on the fact that the delivery and pickup services can be integrated into the same routes, since a vehicle can possibly continue to serve pickup customers after delivery as long as enough vacant capacity is available. Therefore, with flexible and effective utilization of vehicle capacity, the VRPB is widely studied and applied in logistics [2]. As a motivating example, the supermarkets and shops in grocery logistics can be viewed as linehaul customers, and grocery suppliers can be viewed as backhaul customers, where optimizing the travel cost for this supply chain could be essentially described and modeled as a VRPB.

Formally, a *traditional* VRPB prescribes the routes of pickup and delivery to satisfy the demands at (linehaul and backhaul) customer locations, which aims to minimize the total distance while satisfying the following constraints [3]: 1) each vehicle route starts and ends at a depot; 2) each linehaul (backhaul) customer is visited exactly once; 3) the total demand of linehaul (or backhaul) customers in a single route does not exceed the vehicle capacity; and 4) in each route, linehaul customers need to be served before backhaul customers. Typically, the traditional VRPB is featured by the priority constraint in 4), which stipulates that services to linehaul customers must precede those to backhaul customers

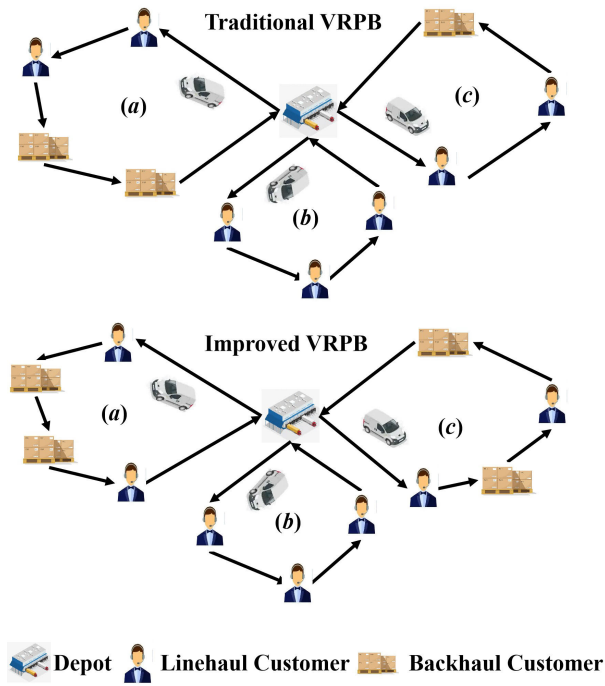


Fig. 1. Examples of the traditional and improved VRPB. Despite the solutions in both problems may contain tours, which only comprise linehaul customers (e.g., tour (b) in two subfigures), the main difference between two problems is that linehaul customers must be visited before backhaul customers in each tour for the traditional VRPB (e.g., routes (a) and (c) in the top subfigure), while this priority constraint is discarded for the improved VRPB (e.g., routes (a) and (c) in the bottom subfigure).

(if any) in the same route. The purpose of this priority constraint is to avoid the potential delay caused by the (frequent) rearrangement of loads in vehicles, e.g., rearranging the loads to make it handy for the next customer. However, the priority constraint may cause lower utilization of the vehicle capacity or longer detours regarding the overall task for serving linehaul and backhaul customers. In contrast to the traditional VRPB, the *improved* VRPB has emerged in recent years, which allows either successive or alternative pickup and delivery in a route, as long as the vehicle capacity constraint is satisfied. Rather than overemphasizing on the priority constraint, the improved VRPB potentially avoids the detours by utilizing the vehicle capacity in a more flexible yet efficient way. We would like to note that the studies of either VRPB variant are meaningful, and each has its own merit and demerit, where two exemplary instances for them are visualized in Fig. 1.

Both the traditional and improved VRPB variants belong to NP-hard combinatorial optimization problems (COPs), given that the original VRP is already of NP-hardness. On the one hand, the exact algorithms have the exponential computational complexity for the worst case, which renders them unsuitable for solving large-scale or complex VRPs [4]. On the other hand, (meta-)heuristic algorithms are able to attain desirable solutions in a reasonable time and have been widely applied to solve VRPs in reality, such as genetic algorithm (GA) [5], [6], large neighborhood search (LNS) [7], and so on. However, these heuristic algorithms are usually developed based on handcrafted rules or massive domain expertise, which may limit the eventual performance. Recently, there is a growing trend toward exploiting deep reinforcement learning (DRL) to automatically learn or discover the rules or heuristics for solving VRPs [8], [9], [10], which delivers *neural* heuristics

that are superior to many conventional heuristics. Neural heuristics are usually able to leverage the pattern among VRP instances and efficiently explore the combinatorial solution space with the power of parallel computing on GPU [11]. Given its success in solving VRP variants [9], [12], [13], such as traveling salesman problem (TSP) and capacitated VRP (CVRP), we aim to expand the capability of neural heuristic to solve VRP with backhauls (VRPBs). To the best of our knowledge, this is the first attempt for neural heuristics to solve VRPB based on DRL, which is nontrivial, since VRPB is more challenging yet practical.

In particular, in this article, we propose a DRL-based neural heuristic to solve the two VRPB variants. First, we represent the VRPB over a graph [14] and cast the solution construction as a Markov decision process (MDP). Then, we exploit an encoder-decoder policy network to construct the solution by sequentially selecting a pickup or delivery node at each step, where we devise a two-stage attention-based encoder to explicitly differentiate all types of nodes (i.e., linehaul and backhaul customers, and the depot). In specific, we leverage a self-attention to capture the relationship among all the nodes at the first stage and design a heterogeneous attention to explicitly capture the relationship between the linehaul and backhaul nodes at the second stage. Afterward, we also exploit respective masking schemes to ensure the feasibility of solutions for the two VRPB variants. Finally, we train the policy network on both traditional and improved VRPB variants using the REINFORCE algorithm, which deliver the eventual neural heuristic. Extensive results on randomly generated instances and benchmark instances show that our neural heuristic could yield high-quality solutions in short computational time and perform favorably against both the conventional and neural heuristics baselines. In the meantime, our neural heuristic exhibits a desirable capability of generalization to different problem sizes and distributions, which significantly outperforms the baselines in most cases.

In the rest of this article, Section II reviews the related literature, and Section III presents the formulations of the two VRPB variants. Then, Section IV elaborates the proposed neural heuristic based on DRL, and Section V exhibits the experimental results. Finally, Section VI concludes this article.

II. RELATED WORKS

The VRPB is primarily featured by two types of customer nodes, i.e., linehaul (delivery) nodes where the customers will receive goods and backhaul (pickup) nodes where the goods are collected from the customers and transported to the depot. According to whether the delivery needs to precede pickup, the problems are generally categorized into the traditional VRPB and the improved VRPB. For each variant, we review the conventional exact and heuristic methods, respectively. We also review the existing neural heuristics for VRPs.

A. Traditional VRPB

1) *Exact Algorithms*: Among the very early attempts, Yano et al. [15] introduced an exact algorithm based on the set-covering formulation to minimize the number of vehicles for a special case of the traditional VRPB, where at most four customers were allowed in a route. Toth and Vigo [16] studied the symmetric and asymmetric versions of VRPB, which were modeled as integer linear programs (ILPs) and solved by a branch-and-bound-based exact algorithm. Mingozzi et al. [17]

modeled the VRPB as a binary integer program and derived its dual problem in the form of relaxed linear program by leveraging different heuristic methods. Then, they exploited the exact solver CPLEX to yield the solution to the dual problem as an effective lower bound of the original VRPB, which were further used to attain the exact solution. In spite of the optimality guarantee, the computational complexity of the exact algorithms usually grows exponentially, and it is hard for them to derive the optimal solutions in a reasonable time for large-sized problems.

2) *Heuristic Algorithms*: Deif and Bodin [18] presented a mathematical description of the traditional VRPB on top of a CVRP model and extended the Clarke and Wright saving (CWS) heuristic to solve the problem [19]. Goetschalckx and Jacobs-Blecha [20] formulated the VRPB as an ILP and solved it using a two-stage method. At the first stage, greedy algorithm was used to group the customer nodes to generate feasible initial solutions. At the second stage, two-opt and three-opt operators were used to iteratively improve the current solutions toward the optimum. Duhamel et al. [21] designed a tabu search heuristic for the VRPB with additional constraints on time windows and precedence of customers, which aimed to minimize both the number of vehicles and the total distance. Toth and Vigo [22] proposed a cluster-first, route-second heuristic for VRPB, which adopted the Lagrangian relaxation to yield an initial solution and then improved it through a local search based on the inter-route and intraroute arc exchanges. Toth and Vigo [23] reviewed the VRPB literature for the first time and introduced exact and heuristic methods to solve the traditional VRPB. Rather than concentrating on one specific problem, Lin–Kernighan–Helsgaun (LKH) [24] was proposed as a generic and powerful heuristic solver for various VRP variants, including the two VRPBs. LKH was developed based on the variable neighborhood search, with a sensitivity analysis to guide the search toward solutions of higher quality, and achieved the state-of-the-art performance on many problems.

B. Improved VRPB

1) *Exact Algorithms*: The improved VRPB was first introduced in [25], which assumed that there was no explicit priority between pickup and delivery services, so as to potentially avoid the underutilized vehicle capacity or the detour in a route. Hernández-Pérez and Salazar-González [26] proposed a branch-and-cut-based exact algorithm to solve the improved VRPB, and Gélinas et al. [27] proposed an approach based on the branch-and-bound and column generation for the improved VRPB with time window constraints. Due to the NP-hard nature, exact algorithms often failed to efficiently solve the improved VRPB, especially in comparison with the heuristic ones.

2) *Heuristic Algorithms*: Golden et al. [25] proposed a construction heuristic for the improved VRPB, which first specified a route through delivery nodes and then iteratively inserted pickup nodes into the route. Similarly, Casco et al. [28] employed the CWS heuristic to construct an initial route for delivery nodes and then exploited an insertion method [25] to allow pickup nodes to be inserted before delivery nodes if the quantity of remaining delivery goods was small. Salhi and Nagy [29] proposed a clustering-based insertion heuristic, which first grouped adjacent nodes into clusters and then inserted them into the route. Regarding

metaheuristics, Ganesh and Narendran [30] proposed a three-phase construction heuristic for the improved VRPB, after which they adopted a GA to refine the constructed route. Paraphantakul et al. [31] exploited an ant colony optimization (ACO) to solve a real-world VRPB with restrictive time windows, the objective of which was to minimize the total travel time. Wassan et al. [32] proposed a reactive tabu search (RTS) algorithm to solve the VRP with mixed deliveries and pickups (i.e., VRPMPD), which refers to the improved VRPB in this article. Avci and Topaloglu [33] proposed an adaptive local search by combining simulated annealing algorithm with variable neighborhood descent to solve the VRPMPD. Regarding the generic LKH solver, as aforementioned, it also solved the improved VRPB, but termed it VRPMPD. For more literature of the improved VRPB or VRPMPD, we refer the readers to [1].

C. Neural Heuristics for VRPs

Recently, there have been growing interests in applying neural heuristics based on DRL to solve VRPs. The most well-known attention model (AM) proposed by Kool et al. [9] was recognized as the first successful application of Transformer-style architecture to TSP and CVRP, which exhibited competitive performance to classic heuristics. On top of AM, Kwon et al. [12] proposed the policy optimization with multiple optima (POMO) to enhance the exploration in training and the augmentation in inference, which is deemed as the state-of-the-art DRL-based construction heuristic for TSP and CVRP. Most of the subsequent neural construction heuristics are developed from AM or POMO. In addition, some works attempt to learn improvement heuristics for routing problems. Wu et al. [13] proposed a DRL framework to learn the node pair selection policy for iteratively improving a solution. Based on it, Ma et al. [34] proposed the dual-aspect collaborative Transformer (DACT) to further boost the performance. Li et al. [35] proposed a feature embedding refiner (FER) with an encoder–refiner–decoder structure to boost the existing pretrained model during the solution sampling stage. Garmendia et al. [36] proposed a graph neural network-based improvement model that encodes crucial information into nodes, edges, or both, which can guide the selection of neighborhood operations in the hill-climbing algorithms. Zhang et al. [37] proposed a DRL method based on meta learning to solve the multiobjective VRPs using the Pareto front. Shao et al. [38] proposed a multiobjective neural evolution algorithm based on decomposition and dominance (MONEADD), which utilizes genetic operations and rewards signals to evolve neural networks for different COPs including TSP. However, no matter the construction or the improvement one, the existing neural heuristics are mostly centered on simple VRP variants, such as TSP and CVRP, which are much less effective in handling the ones with more complex constraints, such as the traditional and improved VRPB variants. To the best of our knowledge, there is scarce literature on learning-based methods for the VRPB. Ghaziri and Osman [39] proposed a self-organizing feature maps (SOFMs) method for solving the traditional VRPB based on unsupervised competitive neural networks. However, the only learning component was used to identify the types of the nodes, rather than selecting the nodes for route construction.

In this article, we propose a DRL-based neural heuristic to solve both the traditional and improved VRPB variants,

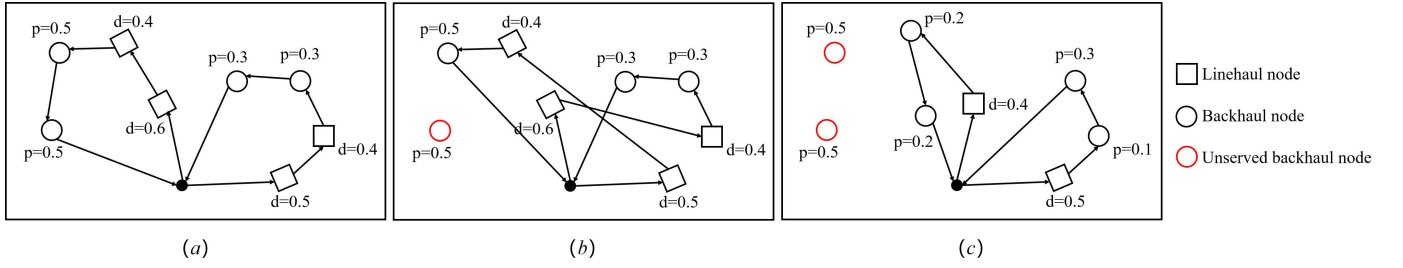


Fig. 2. Exemplary cases in VRPB, where d represents the delivery demand for a linehaul node, p represents the pickup demand for a backhaul node, and the capacity of each vehicle is set to 1. (a) Total demands delivered are more than that of picked up, where the global optimum is possible to achieve. (b) Total demands delivered are also more than that of picked up, but possibly falls into the local optimum with a pickup node unserved. (c) Total demands delivered are less than that of picked up, which also leave some pickup nodes unserved.

which leverages an encoder–decoder structured policy network to automatically learn the route (solution) construction.

III. PROBLEM STATEMENT

In this section, we present the mathematical formulations of both the traditional and improved VRPBs. For the sake of convenience, we list some used notations of sets, parameters, and decision variables in Nomenclature.

A. Mathematical Formulation

Typically, a VRPB instance may comprise a depot and N customer nodes, including M linehaul nodes and $N-M$ backhaul nodes, where the location and demand of each customer node are predefined. Regarding the traditional VRPB, each customer node must be visited exactly once, and the linehaul nodes must be served before the backhaul ones in each single route. The vehicle starts from and finally returns to the depot, and the objective is to minimize the total travel distance with the demands at all customer locations satisfied. Accordingly, the traditional VRPB can be represented as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of nodes, including the depot node, and the linehaul and backhaul nodes (with $\mathcal{V}^*$ denoting all nodes excluding the depot node); $\mathcal{E}$ denotes the arc set including the edges between all nodes in $\mathcal{V}$; and node 0 is used to denote the depot, representing the start point and endpoint. Then, the mathematical formulation of the traditional VRPB is expressed as follows:

$$\min \sum_{k=1}^{\kappa} \sum_{i=0}^N \sum_{j=0}^N c_{ij}^k x_{ij}^k \quad (1)$$

$$\text{s.t.} \sum_{i=0}^N \sum_{k=1}^{\kappa} x_{ij}^k = 1 \quad \forall j \in \mathcal{V}^*, k \in \kappa \quad (2)$$

$$\sum_{i=0}^N x_{ij}^k - \sum_{i=1}^n x_{ji}^k = 0 \quad \forall i \in \mathcal{V}^*, k \in \kappa \quad (3)$$

$$\sum_{i \in \mathcal{B}} \sum_{j \in \mathcal{L}} \sum_{k \in \kappa} x_{ij}^k = 0 \quad (4)$$

$$\sum_{j \in \mathcal{B}} \sum_{k \in \kappa} x_{0j}^k = 0 \quad (5)$$

$$\sum_{i=0}^N u_{ji}^k - \sum_{i=0}^n u_{ij}^k = q_j \quad \forall j \neq 0 \quad (6)$$

$$u_{ij}^k \leq D x_{ij}^k \quad \forall i, j \in \mathcal{V}, k \in \kappa \quad (7)$$

$$x_{ij} \in \{0, 1\}, u_{ij} \geq 0 \quad \forall (i, j) \in \mathcal{E}, k \in \kappa. \quad (8)$$

In particular, the objective function (1) is to minimize the total travel cost in terms of distance. Equation (2) ensures that each node (except the depot) is visited exactly once. Equation (3) ensures the conservation of flow, where a vehicle arrives at a node and also departs from that node. Equations (4) and (5) describe the priority rule in the traditional VRPB, where the delivery nodes should come before the pickup nodes. Specifically, (4) ensures that after visiting a backhaul node, the vehicle is not allowed to directly visit a linehaul node. Equation (5) ensures that the vehicle cannot directly visit a backhaul node after departure from the depot. Equation (6) ensures that the available vehicle capacity is decreased or increased according to the customer demand at each node. Equation (7) ensures that the vehicle capacity cannot be exceeded by the total demand in a single route. Regarding the improved VRPB, it eliminates the priority constraint in (4) and keeps other constraints unchanged.

B. Relaxation

The above constraints are reasonable in theory. However, in reality, they might be unable to fully meet, especially due to the unpredictable ratio of the linehaul to backhaul customers or the uncertain quantity of demand at each customer. Also, the following cases may always happen.

- 1) The demand quantity of linehaul nodes is more than (or equal to) that of backhaul nodes. In this case, the global optimum could be possibly attained in theory if the exact algorithm is used, where an example is displayed in Fig. 2(a). However, pertaining to the heuristics, they may not guarantee that the priority constraint [i.e., (4) and (5)] is always satisfied due to their limited capability in handling complex constraints. Typically, they may allow the violation of the constraints but add corresponding penalty in their objective or utility function [24]. Thus, the resulting solution may not fully comply delivering first and picking up later and fall into a local optimum, where an example is displayed in Fig. 2(b).
- 2) The demand quantity of linehaul nodes is less than that of backhaul nodes. In this case, the delivery and pickup demands allocated to each route cannot meet the priority requirements, and leave some pickup nodes unserved, where an example is displayed in Fig. 2(c).

Given the above cases, we relax (4) and (5) for the two VRPB variants and allow it to only pick up goods in the last few routes, if all linehaul nodes have been served while some backhaul nodes still remain. In particular, we set vehicles to

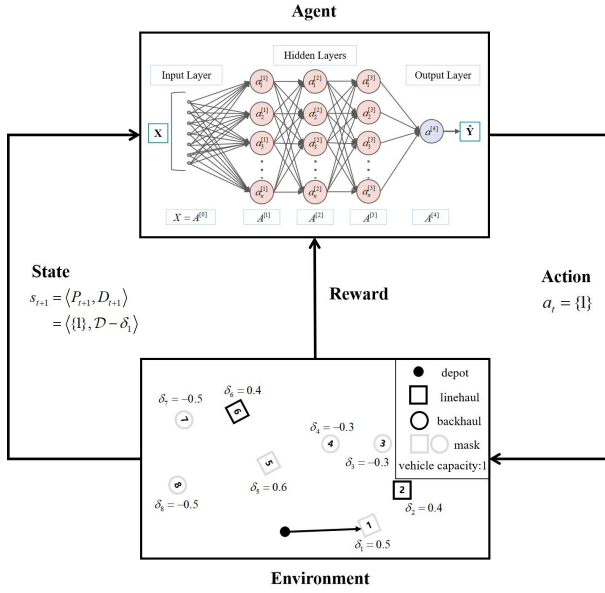


Fig. 3. Schematic of using DRL to solve the VRPB, where we take the traditional VRPB with one track computation as an example. The figure shows the learning process when linehaul node 1 has been selected, and linehaul nodes 2 and 6 are available for the next selection, while linehaul node 5 is masked given vehicle capacity 1. All the backhaul nodes are masked at this step due to the inherent constraint in the traditional VRPB. The state or its representation is updated and fed into the neural network to select the next node.

start with fully loaded capacity. For the traditional VRPB, after all the goods are delivered in a single route, or the remaining goods is not sufficient for a delivery node, the (empty) vehicle is allowed to serve backhaul nodes. For the improved VRPB, the vehicle is allowed to freely serve a backhaul node in a route as long as there is sufficient capacity.

IV. METHODOLOGY

In this section, we present the details of our DRL-based neural heuristic, the overall diagram of which is illustrated in Fig. 3. We first cast the solution construction for VRPB as an MDP. Then, we propose an encoder-decoder-based neural network to parameterize the policy, as depicted in Fig. 4. In the encoder, a two-stage attention is designed to learn the informative embedding, which includes a self-attention with respect to all nodes at the first stage, and a heterogeneous attention with respect to the linehaul and backhaul nodes at the second stage. In the decoder, we apply customized masking schemes to ensure the feasibility of the solutions while deriving the probability for node selection pertaining to the two VRPB variants. Finally, the policy parameters are optimized by using the REINFORCE algorithm [40].

A. RL Formulation

The solution (route) construction for VRPB is essentially a sequential decision-making process [41], and at each step, a node is selected and added to the current partial route, until a complete route is formed. Inspired by POMO [12], rather than only employing one start node, we leverage multiple tracks of sampling and training to enhance the solution diversity, where each track starts with a linehaul node (after the depot node). Since those tracks are functionally homogeneous, we focus

on the description of one track for brevity. In particular, the solution construction from one start node could be modeled as an MDP with five tuples $\langle S, A, \tau, r, \pi \rangle$, which includes the state space S , action space A , state transition τ , reward function r , and policy π , that is parameterized as a neural network π_θ .

1) *State*: State $s_t = \langle P_t, D_t \rangle$ ($s_t \in S$) consists of two elements. The first element P_t represents the partial solution (route), including all nodes visited till time step t . The second element D_t represents the vehicle load at time step t . When $t = 0$, P_0 only contains the depot node, and D_0 is equal to the maximum vehicle capacity $\mathcal{D}$, which means that the vehicle starts at the depot with full load.

2) *Action*: The action a_t ($a_t \in A$) at time step t is represented by a node to be visited or served, i.e., $a_t = v_j$, which is sampled based on policy π and state s_t , i.e., $a_t \sim \pi(a|s_t)$. The nodes that have already been visited are dynamically masked in the environment to avoid solution infeasibility. Then, the sampled actions will constitute the (partial) solution.

3) *Transition*: The next state s_{t+1} is resulted by performing the sampled action to the current state, i.e., $s_{t+1} = f(a_{t+1}, s_t)$. In specific, the transition rule appends the newly selected node to the previously obtained partial solution and updates the remaining vehicle capacity or load as well, i.e., $P_{t+1} = \{P_t; \{v_j\}\}$, $D_{t+1} = D_t - \delta_j$, where δ_j represents the demand at node j accessed at time step t . Note that, δ_j could be either positive or negative according to the type of node j .

4) *Reward*: Our goal is to minimize the route length $L(\pi)$ after a complete route is constructed. Accordingly, the reward function is defined as $R(\pi) = -L(\pi)$. Such a reward favors the actions, which help deliver shorter routes, and thus, the policy will be trained to yield high-quality solutions (routes).

5) *Policy*: The probability of constructing a complete solution is represented by a probability chain rule as follows:

$$\pi(a|s) = \prod_{t=0}^T \pi(a_{t+1}|a_{0:t}, s_t) \quad (9)$$

where π refers to the policy with the trainable parameter θ . At each time step of solution construction for the VRPB, the policy outputs the probabilities of nodes to be selected, based on which a node is sampled, until a complete route is produced, i.e., $a = \{a_0, a_1, \dots, a_T\}$, where $a_0 = v_0$ and T is the maximum time step to construct the solution. Specially, $\pi(a_1|a_{0:0}, s_0) = \pi(a_1|a_0, s_0)$ when $t = 0$. Next, we elaborate how to parameterize the policy π as an encoder-decoder-based neural network π_θ , which can be used to solve both the traditional and improved VRPB variants.

B. Encoder

Our encoder mainly comprises a two-stage attention layer and a feedforward (FF) layer. The former is used to capture the relation among homogeneous nodes and heterogeneous nodes, respectively, and the latter is used to evolve the node embeddings to be more informative, so as to facilitate the subsequent route construction by the decoder.

We take $\mathcal{V} = \{v_0, v_1, \dots, v_N\}$ with $N + 1$ nodes in a VRPB instance as the input. Concretely, $v_i = (x_i, \delta_i)$, where x_i represents the 2-D coordinates of node i , and δ_i ($0 < |\delta_i| < 1$) is the normalized demand. We set $\delta_i > 0$ for a linehaul node, and $\delta_i < 0$ for a backhaul node. When the vehicle starts from the depot, it is fully loaded with the capacity $\mathcal{D}$.

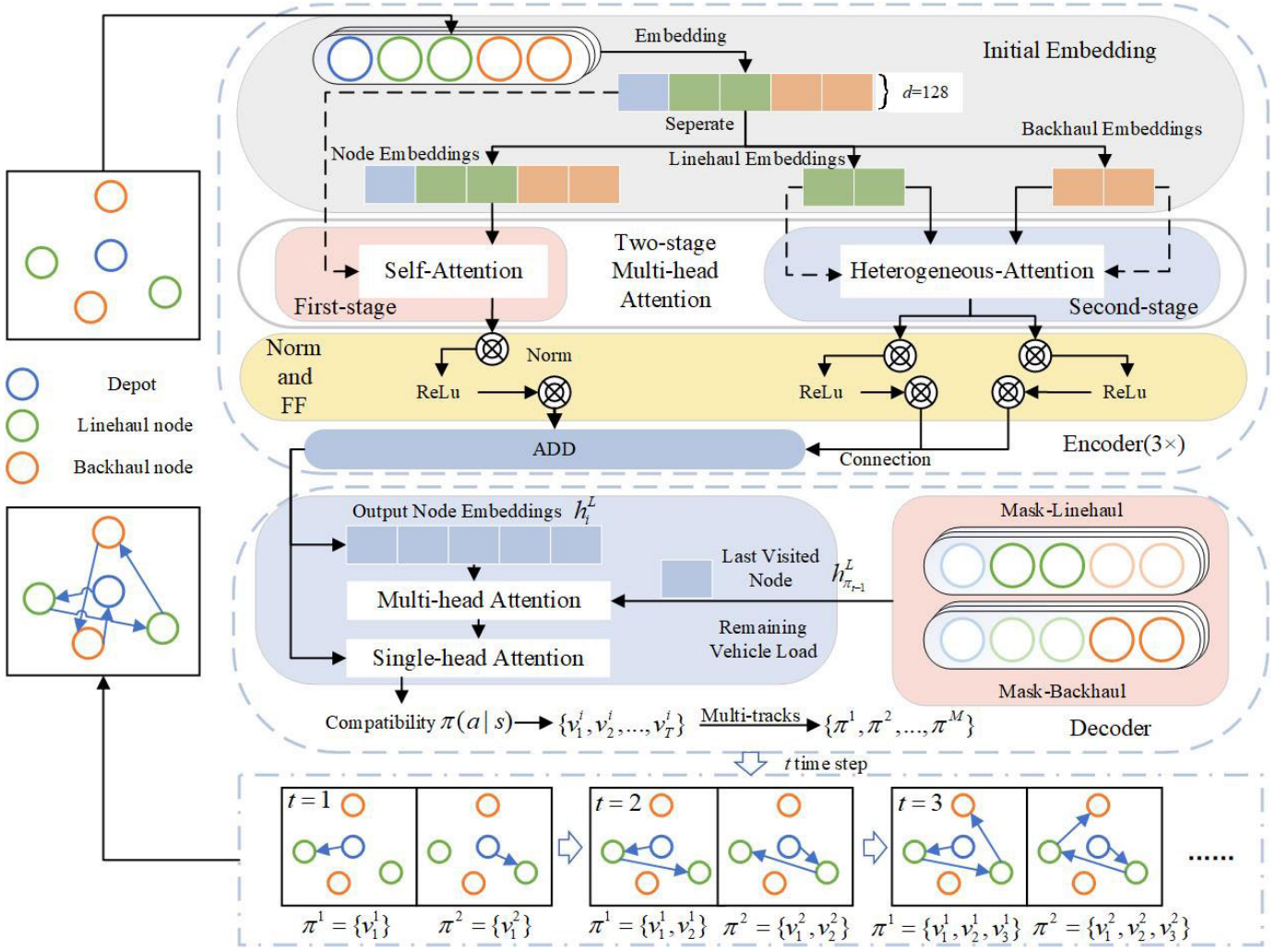


Fig. 4. Overall architecture of our encoder-decoder-based policy network, where we take the traditional VRPB as an example and illustrate a diagram of parallel computation with two tracks ($M = 2$).

The encoder network first converts the coordinates of each node into a d_h -dimensional ($d_h = 128$) initial embedding $h_i^{(0)}$ through a linear projection. Then, the embeddings are processed by L ($L = 3$) attention layers, which share the same structure but different parameters. The function of the attention layer is to realize the information transmission between nodes and, thus, enhance the node representations. Each attention layer is composed of two sublayers, including a two-stage attention layer that performs message passing between nodes, and an FF layer that is shared with each node to evolve their embeddings in parallel.

1) *Two-Stage Multihead Attention Layer*: In the classic VRP variants, such as TSP and CVRP, most of the nodes are homogeneous (except the depot node). Therefore, the (multihead) self-attention is always exploited to capture the relationship among the nodes, which is usually sufficient to achieve favorable global performance [9]. However, in VRPBs, there are primarily two types of heterogeneous nodes (except the depot node) with different attributes, where the self-attention alone may become less effective in yielding informative embedding for the nodes, thus leading to undesirable performance. To better capture both the relationships within the homogeneous nodes and across the heterogeneous

nodes, we design a two-stage multihead attention mechanism. Before introducing the details, we first pick the respective initially yielded embeddings for the depot node, the linehaul nodes, and the backhaul nodes and then concatenate them together as follows:

$$h_i^{(0)} = [h_{\text{depot}}, h_{\alpha}, h_{\beta}] \quad (10)$$

where i represents the node index, h_{depot} represents the depot node, h_{α} represents the linehaul nodes, h_{β} represents the backhaul nodes, and “,” means the concatenation operation.

- 1) At the first stage, there might be a connection between any two nodes in a VRPB instance, such as depot node and (linehaul or backhaul) customer node, linehaul node and linehaul node, linehaul node and backhaul node, and backhaul node and backhaul node, which implies a potential correlation between any two nodes. Therefore, we take the graph of the VRPB instance as a whole and exploit the multihead self-attention to capture the relationships among all the nodes. Specifically, the number of heads is set to $m = 8$, and the dimension of each head is the same as that of the initial node embedding (i.e., $d_h = 128$). For each node in the sequence, corresponding vectors will be created

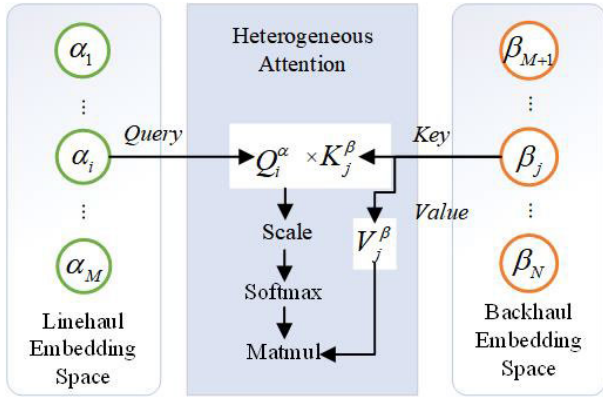


Fig. 5. Our heterogeneous attention, where α and β represent two different types of nodes (i.e., linehaul and backhaul nodes).

to attain the *query* Q and *key-value* (K - V) pair with the dimensions of d_q , d_k , and d_v , respectively, which are expressed as follows:

$$Q_i = W^Q h_i^{(l-1)}, \quad K_i = W^K h_i^{(l-1)}, \quad V_i = W^V h_i^{(l-1)} \quad (11)$$

where $d_q = d_k = d_v = d_h/m$; $h_i^{(l-1)}$ represents the node embedding in the attention layer $l-1$ ($l \in \{1, \dots, L\}$); i refers to the node index; and $W^Q, W^K \in \mathbb{R}^{d_h \times d_k}$, and $W^V \in \mathbb{R}^{d_h \times d_v}$ all represent trainable weight matrices.

To measure the correlation between node i and j , the self-attention first applies a dot product of *query* and *key* of the two nodes and then performs scaling and *softmax* operation before multiplying it with the *value*. The calculation process is highly optimized via matrix multiplication (simplified as MatMul in Fig. 5). Each of the multiple heads executes those procedure in parallel to attain the output attention score as follows:

$$h_o^{(l)} = \text{softmax}\left(\frac{Q_i \times (K_j)^T}{\sqrt{d_k}}\right) V_j. \quad (12)$$

- 2) At the second stage, to further identify the relationship between linehaul nodes and backhaul nodes, we design a heterogeneous attention to strengthen the representations of the two different types of nodes. Given that those nodes still share the same categories of features, although the demand has opposite meanings, we exploit a dual structure in our heterogeneous attention for the two types of nodes, as illustrated in Fig. 5.

Our heterogeneous attention is also of multiheads. The calculation of *query* and *key-value* pair is similar to that of the preceding self-attention, which are expressed as follows:

$$Q_i^\alpha = W^{Q_\alpha} h_{\alpha_i}^{(l-1)}, \quad K_i^\alpha = W^{K_\alpha} h_{\alpha_i}^{(l-1)}, \quad V_i^\alpha = W^{V_\alpha} h_{\alpha_i}^{(l-1)} \quad (13)$$

$$Q_j^\beta = W^{Q_\beta} h_{\beta_j}^{(l-1)}, \quad K_j^\beta = W^{K_\beta} h_{\beta_j}^{(l-1)}, \quad V_j^\beta = W^{V_\beta} h_{\beta_j}^{(l-1)} \quad (14)$$

where $h_{\alpha_i}^{(l-1)}$ represents the linehaul node embedding in the attention layer $l-1$ ($l \in \{1, \dots, L\}$) and α_i refers to the linehaul node. Similarly, $h_{\beta_j}^{(l-1)}$ represents the backhaul node embedding, and β_j refers to the backhaul node.

Regarding the attention score, different from the preceding self-attention, we specify that the two nodes are of different types when calculating the correlations in our heterogeneous attention. In particular, as illustrated in Fig. 5, when calculating the attention score for a linehaul node, the key and the value should be of a backhaul node. Similarly, it applies the other way round when calculating the attention score for a backhaul node. Accordingly, the scores of such heterogeneous attention in each head for the two types of nodes are expressed as follows:

$$h_\alpha^{(l)} = \text{softmax}\left(\frac{Q_i^\alpha \times (K_j^\beta)^T}{\sqrt{d_k}}\right) V_j^\beta \quad (15)$$

$$h_\beta^{(l)} = \text{softmax}\left(\frac{Q_j^\beta \times (K_i^\alpha)^T}{\sqrt{d_k}}\right) V_i^\alpha. \quad (16)$$

2) *FF Layer and Others*: The capability of the encoder can be further enhanced by appending an FF layer. In the FF layer, the neurons only receive the signals from the preceding layer and pass them to the next layer in the forward direction. The ReLu activation is also adopted to calculate the output as follows:

$$\text{FF}(h_i^{(l)}) = \max(0, h_i^{(l)} W_1 + c_1) W_2 + c_2. \quad (17)$$

In addition, skip connection [42] and 1-D instance normalization (IN) [43] are also applied to each sublayer. Skip connection is used to prevent the degradation of accuracy saturation caused by depth increase of the neural network, which is expressed as $F(x + \text{sublayer}(x))$. IN is mainly used to accelerate the convergence and keep the independence between instances. Accordingly, the eventual node embedding is calculated as follows:

$$h_i^L = h_i^{(L)} + \text{Concat}(h_\alpha^{(L)}, h_\beta^{(L)}) \quad (18)$$

where α and β refer to the two different types of nodes. Apart from the node embedding, the encoder also calculates the graph embedding $\bar{h}$ of the input graph, which is the average of the node embedding and expressed as follows:

$$\bar{h} = \frac{1}{N} \sum_{i=1}^N h_i^L. \quad (19)$$

Note that both node embedding h_i^L and graph embedding $\bar{h}$ are used as inputs to the subsequent decoder.

C. Decoder

After receiving the output from the encoder, the decoder calculates the node selection probability through the decoding process to determine the node to be visited at each step. In this process, the decoder combines the graph embedding $\bar{h}$ from the encoder, the last visited node $h_{\pi_{t-1}}^L$ in the current partial solution, and the vehicle load D_t as the vector $h_t^{(c)}$, which represents the *context* as follows:

$$h_t^{(c)} = [\bar{h}, h_{\pi_{t-1}}^L, D_t]. \quad (20)$$

In our decoder, it leverages a multihead attention layer and a single-head attention layer to facilitate the node selection and route construction.

Algorithm 1 Training of Our Neural Heuristic

Input: Instance with a set of nodes $\mathcal{V}$, numbers of epochs E , number of batches B , intermediate step limit per batch (after all nodes that meet the priority constraint are completed) ϕ , numbers of linehaul nodes per instance M , step limit T , size of each instance N , the policy network π_θ^λ for subtrack λ with trainable parameters θ .

Output: A trained policy for route construction.

```

1 Randomly initialize the policy parameters  $\theta$ ;
2 for  $epoch = 1, 2, \dots, E$  do
3   Generate  $B$  instances randomly;
4   for  $batch = 1, 2, \dots, B$  do
5     Retrieve batch,  $t \leftarrow 0$ ;
6     Select multiple start nodes  $\{a_1^1, \dots, a_1^\lambda, \dots, a_1^M\}$ ;
7     for  $\lambda = 1, 2, \dots, M$  do
8       while  $t < T$  do
9         Take an action  $a_t^\lambda \sim \pi(a|s_t^\lambda)$  from  $\mathcal{V}$ ;
10        while  $t > \phi$  do
11          Take an action from remaining nodes;
12        end
13        Get reward  $r_t^\lambda$  and next state  $s_{t+1}^\lambda$ ;
14         $t \leftarrow t + 1$ ;
15      end
16       $R^\lambda = \sum_{t=0}^T r_t^\lambda$ ;
17    end
18     $R(\pi) = -\max(R^\lambda)$ ;
19    Update baseline  $b(s)$  according to Eq. (26);
20    Calculate gradient  $J(\theta)$  according to Eq. (25);
21    Update  $\theta$  by  $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$ .
22  end
23 end

```

1) *Multihead Attention Layer*: The multihead attention in the decoder is mainly used to aggregate the information of the context node. Different from the multihead attention in the encoder, the *query* here comes from the context vector, and the *key/value* comes from the node embedding output by the encoder. The calculation is expressed as follows:

$$h_t^c = \text{MHA}\left(W_{(c)}^Q h_t^{(c)}, W_{(c)}^K h_i^L, W_{(c)}^V h_i^L\right) \quad (21)$$

where $W_{(c)}^Q$, $W_{(c)}^K$, and $W_{(c)}^V$ are trainable parameter matrices.

2) *Single-Head Attention Layer*: The single-head attention mechanism in the decoder is mainly to derive node selection probability. Here, only the feature vectors query and key are used in the calculation as follows:

$$Q_t^c = W_c^Q h_t^c, \quad K_i^c = W_c^K h_i^L \quad (22)$$

where query is from the context node and key is from the node embedding output by the encoder. Based on the dot product of the above two vectors, the node compatibility will be derived. In order to promote exploration and improve marginal performance, we use the tanh function to adjust node compatibility to the range of $[-C, C]$ (i.e., $C = 10$) as follows:

$$u_i = C \bullet \tanh\left(\frac{Q_t^c K_i^c}{\sqrt{d_k}}\right). \quad (23)$$

To ensure the feasibility of the solution, we set the mask with a compatibility score of negative infinity at the end of decoder. There are four types of infeasible nodes that need to be masked in the *traditional* VRPB, i.e., the following hold: 1) customer nodes that have been visited; 2) customer nodes that cannot be reached due to vehicle capacity limitations; 3) (linehaul or backhaul) customer nodes that may violate the priority constraint at a step; and 4) the depot node when there are still remaining customer nodes that the vehicle is able and allowed to visit. The *improved* VRPB only removes the priority constraint in 3), while other three types of nodes still need to be masked.

After the node compatibility is accordingly masked given the feasibility requirement, the decoder calculates the final probability vector by using the *softmax* function as follows:

$$\pi(a|s) = \frac{\exp(u_i)}{\sum_i \exp(u_i)}. \quad (24)$$

Then, the policy will output the node to be visited as the one sampled according to the probability.

D. Policy Optimization

To boost the performance of our neural heuristic, we leverage multiple start nodes in a VRPB instance to train our policy in multiple tracks, where each track starts from a different linehaul node (after departure from the depot). Therefore, for an instance with $N + 1$ nodes (including one depot node and M ($M < N$) linehaul nodes), the number of exploration tracks is M . The training of the policy is performed using the REINFORCE algorithm [40]. In each track, to maximize the reward (i.e., minimize the route length), we use the gradient ascent to optimize the policy parameter θ as follows:

$$\nabla_\theta J(\theta) = \frac{1}{M} \sum_{\lambda=1}^M (R(\pi^\lambda) - b(s)) \nabla_\theta \log \pi_\theta^\lambda(a|s) \quad (25)$$

where λ refers to the index of solution tracks and M tracks for a problem instance use the same $b(s)$, called the *shared* baseline and expressed as follows:

$$b(s) = \frac{1}{M} \sum_{\lambda=1}^M R(\pi^\lambda). \quad (26)$$

This baseline is used to stabilize the training, thus also reducing the computation overhead. If the current policy network is obviously superior to the baseline network, then the parameter of the latter will be replaced with the former one. After dozens of training steps in each epoch, the trained policy network could produce high-quality solutions. The overall procedure of the training is presented in Algorithm 1.

V. EXPERIMENTS

In this section, we evaluate the proposed neural heuristic on both the traditional and improved VRPBs. We first compare our method with the conventional and neural baselines and then evaluate its generalization to different problem sizes and distributions. Finally, we also test our method on benchmark instances to further verify its efficacy.

A. Experimental Settings

1) *Datasets*: Our experiments are mainly conducted on two types of datasets, i.e., the randomly generated one and the benchmark one. Regarding the former dataset, we follow the convention [9], [12], [13], [34] and generate random instances of four sizes, with the number of customer nodes $N = 20, 50, 100$, and 150 , respectively. Concretely, we produce coordinates of customer nodes by uniformly sampling them from a square $[0, 1] \times [0, 1]$ and set the origin as the depot node. We produce linehaul demands by uniformly sampling them from $[1, 10]$ and backhaul demands from $[-10, -1]$. We set the ratio of linehaul to backhaul nodes to 1:1. Note that since demands of linehaul and backhaul nodes are produced randomly, it cannot guarantee that the total demand of linehaul nodes is more than that of backhaul nodes. Regarding the latter dataset, we employ instances used in [44] for the traditional VRPB and the ones used in [45] for the improved VRPB, which are totally different from the randomly generated ones, in terms of the node distribution and the quantity of demands. However, we will show that even our policy trained only on random instances can be generalizable to these hard instances with desirable performance.

2) *Training and Testing*: For either VRPB variant, we train the policy network for 100 epochs, with 10000 batches in each epoch and 64 instances (generated on the fly) in each batch. We set the learning rate to 0.0001. After each training epoch, we generate 10000 random instances and use them as a validation set. As mentioned previously, we will test our neural heuristics on both random instances and benchmark instances. Regarding the random testing set, we only generate 200 instances for each size and each variant, since the conventional baseline may take a long time to solve even a single instance. Also, we adopt the neural heuristics that are trained only on the random instances to solve all types of testing instances, including the benchmark ones. All the neural heuristics are trained and tested on a server equipped with a GeForce RTX GPU and Intel i7-10700X CPU at 3.80 GHz. For a fair comparison with the conventional baselines pertaining to the runtime, during the testing, all the neural heuristics including ours solve the VRPB instances one by one rather than in parallel and then count the average runtime for solving one instance.

3) *Baselines*: We compare the proposed neural heuristic with various baselines, including the exact solver, conventional heuristics, and other neural heuristics. The details of the baselines are described as follows.

- 1) CPLEX, the well-known commercial exact solver, is used to solve the mixed integer programming (MIP) of the VRPB described in Section III. We adopt CPLEX20.1 [46] to compute the solution to each testing instance, with a maximum time limit of 1 h (or 3600 s).
- 2) LKH-3 [24], a strong conventional heuristic solver for various VRPs, is widely used to benchmark other methods in the literature of VRPs. In particular, LKH-3 uses a penalty function to handle the constraint, and the penalty value depends on the degree of constraint violation. Similar to CPLEX, we allow it to run with a maximum time limit of 1 h, and the number of runs is set to 10.
- 3) GA, a conventional metaheuristic widely used to solve VRPs. We develop the GA based on [47], where we set

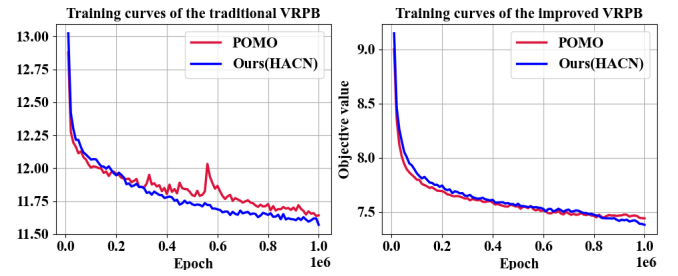


Fig. 6. Exemplary training curves of neural heuristics on both traditional and improved VRPBs with 50 nodes.

the population size, the crossover, and mutation probability to 100, 0.8, and 0.1, respectively. The number of iterations is 100, and the maximum time limit is the same as CPLEX and LKH-3, which is 1 h.

- 4) Adaptive LNS algorithm (ALNS), another widely used conventional metaheuristics for solving VRPs. We develop the ALNS based on [48]. In particular, we set the destroy degree to 0.2, and the destroy and repair operators are selected according to the weight probability. We set the initial temperature, decrease rate, and maximum number of improvements to 30, 0.94, and 200, respectively.
- 5) AM, a well-known DRL-based neural heuristic for VRPs [9]. We apply two different decoding schemes in AM, i.e., *sampling* and *greedy*, respectively. The former samples 1280 solutions using the learned policy and outputs the best one, the eventual solution quality of which is usually better than that of the latter. We adapt AM to solve both the traditional and improved VRPB variants by modifying the masking schemes accordingly. Other settings follow its original paper.
- 6) POMO, a strong neural heuristic developed on top of AM with sampling strategy, is further enhanced through exploration of multiple tracks (or start nodes) and data augmentation [12]. We also adapt it to solve the two VRPB variants, and other settings follow its original paper.
- 7) FER [35], an encoder-refiner-decoder structure that boosts a given deep model during solution sampling. The encoder-decoder can be adapted from any pretrained neural construction heuristic, such as AM and POMO. Due to the superior performance of POMO, we harness and adapt the POMO for VRPB as the pretrained model, with an iteration limit of 200 for refining. Other settings follow its original paper.

Before we compare those methods in detail, we first show the training curves of the neural heuristics. Since the performance of AM is considerably inferior (as revealed in Tables I and II), we primarily focus on our neural heuristic and the strong neural baseline, i.e., POMO. The training curves of the two methods are displayed in Fig. 6, which are attained based on both traditional and improved VRPBs with 50 customer nodes. It is observed that the objective values (i.e., total route length) for the two methods decrease rapidly at the beginning. Afterward, the training gradually gets stabilized. In addition, the training curve of our method decreased slightly slower than that of POMO in the early phase, but sooner after that, our method achieves the lower objective values.

TABLE I
RESULTS ON THE TRADITIONAL VRPB

Method	$N=20$			$N = 50$			$N = 100$			$N = 150$		
	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)
CPLEX	6.587	0.00%	8.040	14.498	38.34%	3600	26.185	66.94%	3600	—	—	—
LKH-3	6.587	0.00%	0.07	10.48	0.00%	345.06	15.685	0.00%	3600	20.127	0.00%	3600
GA	15.011	127.89%	34.820	107.395	924.76%	61.815	324.921	2086.25%	127.977	445.066	2111.32%	210.295
LNS	10.776	63.59%	0.315	19.991	90.75%	0.434	35.156	124.13%	1.138	41.399	105.69%	2.130
AM(Sampling)	8.418	27.80%	145.066*	18.247	74.11%	913.174*	52.115	232.26%	3600*	78.584	290.44%	3600*
AM(Greedy)	22.973	248.76%	0.128*	51.687	393.20%	0.272*	60.970	288.71%	0.514*	87.420	334.35%	0.647*
POMO	7.774	18.02%	0.160*	14.162	35.13%	0.672*	19.228	22.59%	3.480*	23.762	18.06%	9.779*
FER	8.775	33.22%	31.668*	18.365	75.24%	218.366*	36.984	135.79%	1003.061*	51.334	155.05%	3600*
Ours	6.810	3.39%	0.202*	13.056	24.58%	0.753*	18.573	18.41%	3.748*	22.713	12.85%	10.705*

* For a fair comparison with the conventional methods, during the testing, all the neural heuristics including ours solve the traditional VRPB instances one by one rather than solving them together in parallel, and then record the average runtime for solving one instance.

TABLE II
RESULTS ON THE IMPROVED VRPB

Method	$N = 20$			$N = 50$			$N = 100$			$N = 150$		
	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)
CPLEX	4.448	0.00%	6.514	7.152	0.00%	3600	10.408	0.00%	3600	17.739	35.82%	3600
LKH-3	5.773	29.79%	1.04	9.722	35.93%	1990.59	14.063	35.12%	3600	18.016	37.94%	3600
GA	6.872	54.50%	19.274	76.580	970.75%	282.749	62.282	498.43%	3600	152.720	1069.28%	3600
LNS	7.336	64.93%	0.219	15.597	118.08%	1.296	30.122	189.42%	1.902	36.186	177.05%	3.703
AM(Sampling)	5.010	12.63%	31.114*	8.855	23.81%	69.883*	48.897	369.80%	103.071*	73.858	465.49%	193.332*
AM(Greedy)	10.375	133.25%	0.016*	74.483	941.43%	0.072*	53.960	418.47%	0.087*	86.763	564.29%	0.130*
POMO	5.280	18.71%	0.103*	7.953	11.20%	0.166*	10.914	2.88%	0.286*	13.805	5.70%	0.449*
FER	6.596	48.29%	17.676*	12.849	79.66%	29.851*	24.108	131.63%	63.554*	30.346	132.34%	111.243*
Ours	4.751	6.81%	0.201*	7.662	7.13%	0.255*	10.597	1.82%	0.385*	13.061	0.00%	0.576*

* For a fair comparison with the conventional methods, during the testing, all the neural heuristics including ours solve the improved VRPB instances one by one rather than solving them together in parallel, and then record the average runtime for solving one instance.

B. Comparison Study

The results on the traditional and improved VRPBs are gathered in Tables I and II, respectively. For each method, we record the average of objective values, gaps, and runtime over all the testing instances, respectively. In particular, the gap of a target method is calculated against the best one among all the studied methods who achieved the lowest objective value.

1) *Traditional VRPB*: In terms of the solution quality, CPLEX attained the optimal ones on size $N = 20$. However, as the number of nodes increases, its performance drops significantly given a limited runtime (i.e., 3600 s). For $N = 150$, CPLEX cannot even find a feasible solution due to the huge search space for the NP-hard problems. As a strong conventional heuristic solver, LKH-3 has no obvious performance degradation, as the number of nodes increases, which achieved the best solution for the four sizes at the price of significant increments in runtime. Except the two conventional solvers, our method (denoted as ours) achieves the best solution quality among all the remaining heuristics. Regarding the neural heuristic baselines, the gap of POMO is inferior to our method but much superior to AM, which demonstrates that starting from multiple nodes could potentially improve the eventual

solution quality. For AM, the performance varies significantly with its decoding strategy (greedy or sampling) on small sizes, which verifies the effectiveness of sampling in improving the solution quality. However, AM (sampling) demonstrates overfitting, as the number of nodes becomes relatively large, resulting in a considerable drop in performance. Similarly, FER, developed based on POMO for VRPB, also exhibits overfitting issue, resulting in higher objective value than the original POMO and even worse performance than AM on small-scale nodes. Both AM (greedy) and GA perform inferior on size $N = 20$. When the number of nodes continues to increase, the GA achieves the worst solution quality, which might converge prematurely, likely falling into local optimum. The LNS algorithm performs relatively stable with a better performance than that of AM (greedy) and GA, but much inferior to our neural heuristic.

Regarding the computation efficiency, a time limit of 3600 s is imposed to CPLEX, LKH-3, AM (sampling), and FER whose runtime increases considerably, as the number of nodes grows. Apart from them, the GA consumes the longest runtime, which might be caused by the inefficient search for the hard problem. The LNS algorithm is relatively fast and shows

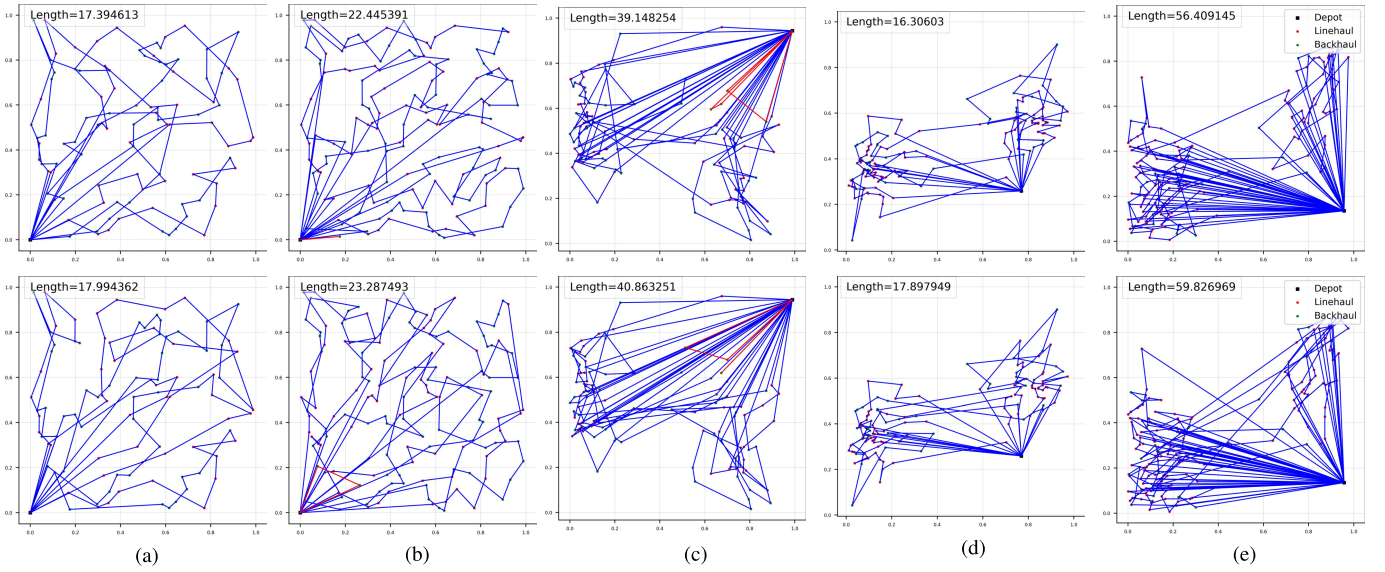


Fig. 7. Detailed results of some exemplary instances for the traditional VRPB. The top row refers to our method with shorter route length, and the bottom row refers to POMO. The red line indicates the tour that is full of backhaul nodes. (a) $N = 100$. (b) $N = 150$. (c) $X - n125 - 50 - k16$. (d) $X - n157 - 66 - k9$. (e) $X - n200 - 80 - k29$.

a linear growth, as the size increases. The runtime of almost all neural heuristics also increases, as the number of nodes grows. Among them, the runtime of our method is slightly longer but roughly the same order of magnitude as that of POMO. Since the solution quality of our method is much better, the increment in runtime is acceptable. In addition, AM (greedy) exhibits the shortest runtime, while its solution quality is poor.

2) *Improved VRPB*: In terms of the solution quality, CPLEX attained the optimal solutions on size $N = 20$ and the best solutions on sizes $N = 50$ and 100 within the time limit, while our method performs the second best. As the size increases up to $N = 150$, our neural heuristic achieves the best solutions, which also exhibits the superiority in solving large-scale problems. Especially, CPLEX cannot obtain a feasible solution for the traditional VRPB of size $N = 150$ (in Table I) within the time limit, but can solve the improved VRPB of the same size (in Table II). This is due to the stricter constraints in the traditional VRPB. To be specific, traditional VRPB requires routes in solutions to follow more restrained permutations that are “first linehaul then backhaul.” In comparison, improved VRPB is more relaxed than traditional VRPB, allowing more feasible permutations in the routes. We also refer to similar problems in the literature and find that the computational complexity of its LP relaxation (linear programming relaxation) of traditional and improved VRPB is approximately featured by $O(N^3)$ and $O(N^2)$, respectively [23], [26]. Among other methods, AM (sampling) achieved superior performance to POMO and only next to CPLEX and our method on size $N = 20$. This implies its applicability in the problem of lower complexity with fewer nodes. Similar to the traditional VRPB, AM (sampling) runs into overfitting on sizes $N = 100$ and 150 , while AM (greedy) also performs significantly inferior. POMO exhibits desirable results again on sizes $N = 50$, 100 , and 150 , but they are still inferior to our method. FER performs better than AM on all sizes, but still exhibits a gap to POMO. Regarding other conventional methods, LKH-3 performs much better than others, but there is still a salient gap to the best solution given the time limit. The results of GA on $N = 20$ are better than

that of LNS, but as the number of nodes grows, premature phenomenon appears quickly for GA. The results of LNS are relatively stable, but the solution quality is much lower than that of our method.

Regarding the computation efficiency, CPLEX, LKH-3, and GA consume the longest runtime up to 3600 s. Among the remaining methods, the neural heuristics including ours are more computationally efficient. Especially, our method takes no more than 1 s to solve an instance in average for all the sizes, which is comparable to POMO and AM (greedy).

3) *Visualization of Exemplary Instances*: We also visualize the detailed results of our method and POMO for some exemplary instances of both traditional and improved VRPBs, which are displayed in Figs. 7 and 8, respectively. Regarding the traditional VRPB, we select the randomly generated instances with $N = 100$ and 150 and the benchmark instances (the ones used in Section V-D) whose sizes and distributions are unseen during training. Regarding the improved VRPB, we select the randomly generated instances with $N = 100$ and 150 , and $N = 200$ and 300 (the ones used in Section V-C) whose sizes are unseen during training, and $N = 150$ with beta distribution (the one used in Section V-C) whose distribution is unseen during training. We can observe that, in spite of the size and distribution, our method could yield different yet better routes than POMO on both VRPB variants, although the two methods are both of neural construction heuristics.

C. Generalization Study

To evaluate the generalization capability of our method, we further test our neural heuristic on cross-size instances and cross-distribution instances, respectively.

1) *Cross-Size Generalization*: In this case, we generate 200 instances of larger size with $N = 200$ and $N = 300$, respectively, and use the pretrained model with $N = 150$ to solve them. Due to the low-computational efficiency and inferior performance of CPLEX, GA, and LKH-3 with the time limit of 3600 s on both problems, we only compare our method with LNS, AM, POMO, and FER. The results

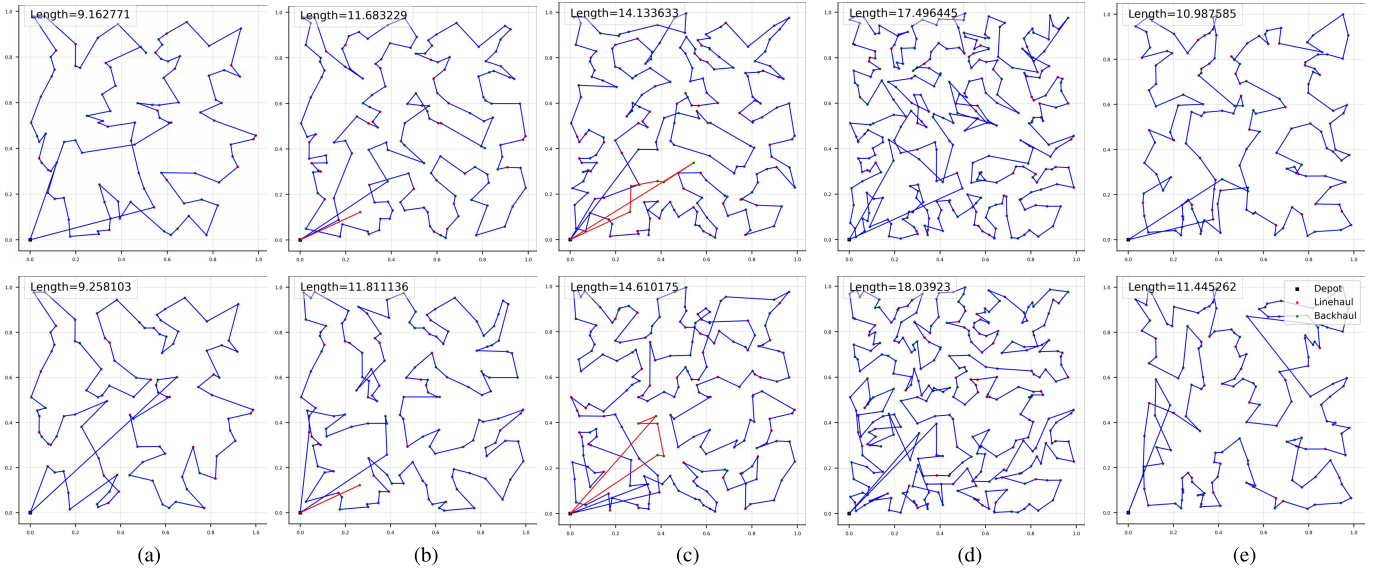


Fig. 8. Detailed results of some exemplary instances for the improved VRPB. The top row refers to our method with shorter route length, and the bottom row refers to POMO. The red line indicates the tour that is full of backhaul nodes. (a) $N = 100$. (b) $N = 150$. (c) $N = 200$. (d) $N = 300$. (e) Beta ($N = 150$).

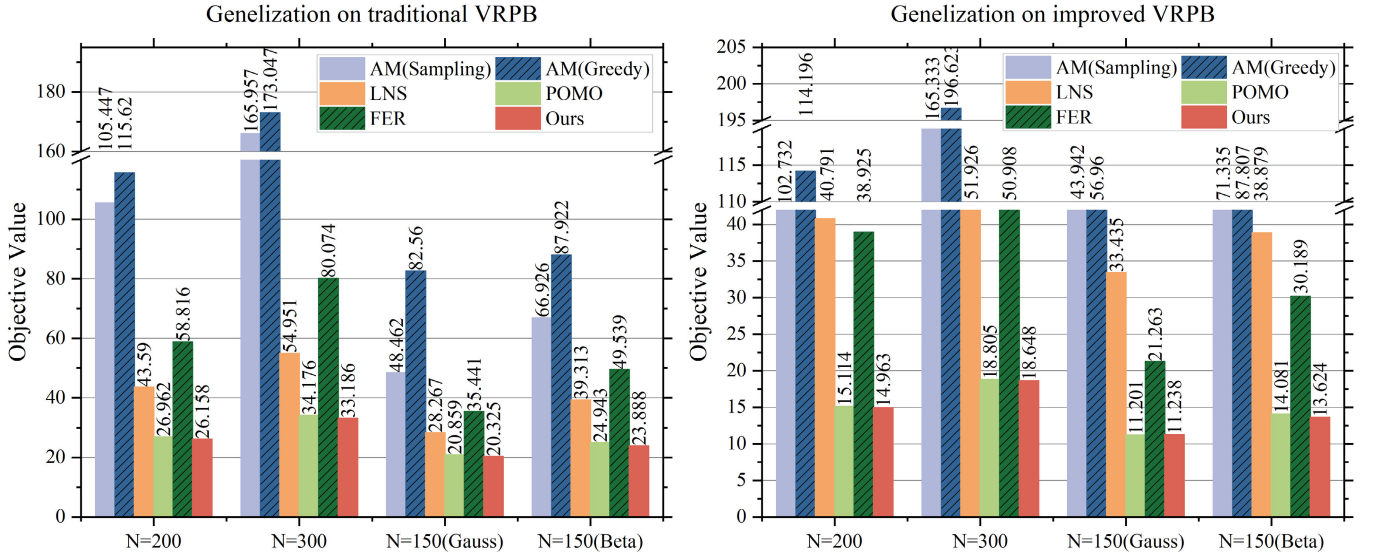


Fig. 9. Generalization to different sizes and distributions.

regarding the average route length are recorded in Fig. 9 for the two VRPB variants. It can be observed that AM performs poorly on larger instances, with sampling decoding achieving lightly better performance than that of greedy decoding. LNS and FER perform moderately well, but the solution quality is still much worse than our method and POMO, with FER being (slightly) inferior to LNS. Although the performance of POMO is strong, our neural heuristic could further boost the solution quality over it, which also verifies the design of our method.

2) *Cross-Distribution Generalization*: In this case, we generate 200 instances with $N = 150$, whose coordinates follow Gaussian distribution and beta distribution, respectively, and use the pretrained model with $N = 150$ whose coordinates follow the uniform distribution to solve them. Regarding the Gaussian distribution, we set the mean of coordinates as 0.5 and the standard deviation as $1/6$, which ensures that 99.7% of nodes fall within the range of (0, 1). Regarding the beta

distribution, we set both two key parameters as 1 to ensure that the mean of node coordinates is 0.5. The results pertaining to the average route length are also recorded in Fig. 9 for the two VRPB variants. It can be observed that the performance of AM with either decoding strategy is still poor, while LNS and FER perform slightly better, all of which are much inferior to POMO and our neural heuristic. Although our neural heuristic is lightly inferior on Gaussian distribution for the improved VRPB, it achieves higher solution quality than POMO in all other cases.

D. Evaluation on Benchmark Dataset

We apply the proposed method to the instances of public benchmark dataset to further verify its generalization performance. In particular, we adopt the benchmark instances in [44] and [45] for the traditional VRPB and improved VRPB, respectively. Note that these instances are completely

TABLE III
AVERAGE RESULTS ON BENCHMARK INSTANCES FOR BOTH VRPB VARIANTS

Method	Traditional VRPB						Improved VRPB				
	BCP_{F2}	GA	LNS	AM (Sampling)	POMO	Ours	RTS	Halse	POMO	FER	Ours
Obj.	21746.318	63067.439	37365.333	76507.788	29094.197	27976.773	257839.933	260697.644	325101.844	525365.156	296103.911
Time(s)	2820.175	98.873	1.363	885.536	15.552	16.469	12.596	1.014	0.178	45.261	0.255

TABLE IV
RESULTS OF DIFFERENT SAMPLING SIZES ON BOTH TRADITIONAL AND IMPROVED VRPBs WITH 100 NODES

Sampling size		320			640			1280			2560		
		Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)	Obj.	Gap	Time(s)
Traditional VRPB	AM	54.911	201.01%	1072.762	53.333	194.74%	2000.346	52.115	189.61%	3600	50.122	178.64%	3600
	POMO	18.497	1.40%	44.021	18.464	2.04%	185.425	18.364	2.05%	811.154	18.221	1.30%	3600
	Ours	18.242	0.000%	52.427	18.095	0.00%	216.114	17.995	0.00%	963.655	17.988	0.00%	3600
Improved VRPB	AM	49.806	440.61%	25.696	49.216	435.60%	51.296	48.897	437.63%	103.071	48.241	434.70%	244.51
	POMO	9.225	0.13%	0.534	9.199	0.11%	0.88	9.159	0.70%	1.451	9.064	0.47%	2.518
	Ours	9.213	0.00%	0.665	9.189	0.00%	1.229	9.095	0.00%	1.877	9.022	0.00%	3.628

different from the ones used in our training, in aspects of node location (or coordinates), customer demand, vehicle capacity, and so on.

1) *Traditional VRPB*: The benchmark instances for the traditional VRPB in [44] are created based on the CVRPLib in [49]. We select all 66 instances with $100 < N \leq 200$, the proportion of linehaul customers in which is 50%, 66%, and 80%, respectively, and solve them using our pretrained model with $N = 150$. Besides the baselines used in this article, we also compare with the BCP_{F2} method proposed in [44], which is an exact algorithm based on branch-cut-and-price. The average objective values and runtime are summarized in Table III. For a full detailed result of each individual instance, please refer to Table I in the supplementary material, where n refers to the number of nodes, k refers to the ideal number of vehicles, and the middle number refers to the proportion of linehaul customers. Since the CPLEX and LKH-3 failed to deliver a solution within 3600 s for a number of instances (i.e., 27.3% and 48.5%), we only reported their detailed results in Table I in the supplementary material rather than the averaged one in Table III.

It can be observed from Table III (left half) that, our method is only next to the BCP_{F2} in terms of solution quality, but consumes significantly shorter time than BCP_{F2} . With comparable runtime, our method achieves better solutions than that of POMO. From Table I in the supplementary material, we can see that when the number of nodes is small, the CPLEX and LKH-3 exhibit obvious advantages, and they alternately obtain the best solution. However, as the number of nodes increases to a certain extent, especially when $N > 180$, our method could basically attain more desirable solutions than them.

2) *Improved VRPB*: The benchmark instances for the improved VRPB in [45] are adapted from the ones in [20]. After removing the duplicate ones, we selected the 45 instances with $25 \leq N \leq 150$, whose proportion of backhaul customers is between 25% and 50%. Then, we use our pretrained model with $N = 150$ to solve them. Apart from POMO and FER, we also compare with two strong

conventional heuristics, who reported the best results for those benchmark instances, i.e., a cluster first-route second heuristic proposed by Halse [45] and a meta-heuristic based on RTS proposed by Wassan et al. [32]. The average objective value and runtime are summarized in Table III. For detailed result of each individual instance, please refer to Table I in the supplementary material, where $\mathcal{V}$ refers to the number of nodes in the instance.

It can be seen from Table III (right half) that RTX and Halse achieved the best and second-best objective values with the runtime the other way round. Our method achieved inferior yet comparable solution quality to them, but consumes much shorter time. Compared with POMO, with similar runtime, our method attained higher solution quality than it. We can see from Table I in the supplementary material that, our method has obtained the best results in 17 of 45 instances with much shorter runtime, which justified its effectiveness in solving instances of various sizes and distributions.

E. Effect of Sampling Size

In Table IV, we analyze the effect of sampling size on the neural heuristics, which are investigated on both traditional and improved VRPBs with 100 nodes. We select the AM (sampling) method and apply the same *sampling* decoding strategy to POMO and our method, so as to increase their running time and improve the solution quality. In particular, we sample 320, 640, 1280, and 2560 solutions using the respective learned policies, but within the maximum time limit of 3600 s. It can be observed that our method achieved superior performance to AM and POMO on all sampling sizes, suggesting the effectiveness of our method. In addition, the performance of POMO and our method with sampling decoding gradually exceeds that with greedy decoding (by referring to the results in Tables I and II) as the sampling size grows, while the run time is also becoming significantly longer. This is due to the fact that more sampling improves solution diversity and quality, but at the price of additional computation.

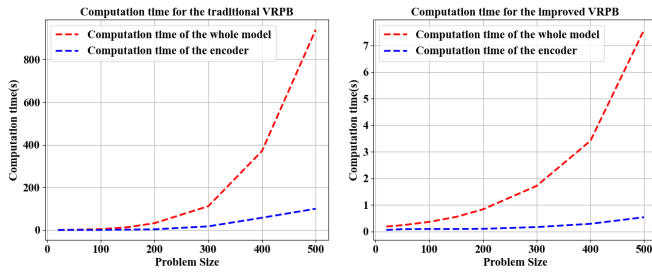


Fig. 10. Average computation time of the whole model and its two-stage encoder over different problem sizes.

F. Complexity Analysis

We finally provide the complexity analysis of the proposed method. We generated 200 instances with $N = 400$ and $N = 500$ and used the instances from Sections V-B and V-C for other sizes. We present the average computation time of our whole model and the two-stage attention-based encoder, as shown in Fig. 10. It can be observed that the computation time of the whole model and its encoder grows quadratically, as the size of the problem increases, which seems to align with the quadratic computational complexity of the attention mechanism [9], [12]. However, compared with the whole model, the computation time of the two-stage encoder and its increasing rate is relatively small, indicating that this newly designed encoder does not introduce much additional computational complexity.

VI. CONCLUSION AND FUTURE WORKS

In this article, we propose a neural heuristic based on DRL to solve two variants of VRPBs, i.e., the traditional VRPB and the improved VRPB, respectively. In our neural heuristic, we exploit an encoder-decoder policy network, where a two-stage attention is proposed in the encoder to learn more informative embedding in the presence of heterogeneous nodes, and a masking scheme is used in the decoder to ensure the feasibility of the solutions. Our method is evaluated on the randomly generated and the benchmark instances, respectively, both of which verified its superiority to either conventional or neural baselines, in achieving higher quality solutions against different sizes and distributions. In the future, we plan to do the following: 1) reduce the computation overhead of our neural heuristic, so that it could solve much larger instances, say the ones with more than 1000 customers; 2) consider heterogeneous vehicles and also explicitly minimize the number of needed vehicles; 3) investigate the problems in the non-Euclidean space; and 4) apply more advanced exploration techniques in DRL to enhance the performance of our model (e.g., [50], [51]).

REFERENCES

- [1] M. J. Santos, P. Amorim, A. Marques, A. Carvalho, and A. Póvoa, "The vehicle routing problem with backhauls towards a sustainability perspective: A review," *TOP*, vol. 28, no. 2, pp. 358–401, Jul. 2020.
- [2] A. Gutiérrez-Sánchez and L. B. Rocha-Medina, "VRP variants applicable to collecting donations and similar problems: A taxonomic review," *Comput. Ind. Eng.*, vol. 164, Feb. 2022, Art. no. 107887.
- [3] P. Toth and D. Vigo, "Vrp with backhauls," in *The Vehicle Routing Problem*. Philadelphia, PA, USA: SIAM, 2002, pp. 195–224.
- [4] P. Augerat, J.-M. Belenguer, E. Benavent, A. Corbéran, and D. Naddef, "Separating capacity constraints in the CVRP using Tabu search," *Eur. J. Oper. Res.*, vol. 106, nos. 2–3, pp. 546–557, 1998.
- [5] W. N. I. WA, M. Shaiful, M. Shamsunarnie, Z. Zainuddin, and M. Fuad, "Genetic algorithm for vehicle routing problem with backhauls," *J. Sci. Technol.*, vol. 4, no. 1, pp. 9–12, 2012.

- [6] D. Whitley, "A genetic algorithm tutorial," *Statist. Comput.*, vol. 4, no. 2, pp. 65–85, Jun. 1994.
- [7] O. Dominguez, D. Guimarans, A. A. Juan, and I. de la Nuez, "A biased-random large neighbourhood search for the two-dimensional vehicle routing problem with backhauls," *Eur. J. Oper. Res.*, vol. 255, no. 2, pp. 442–462, Dec. 2016.
- [8] L. Duan et al., "Efficiently solving the practical vehicle routing problem: A novel joint learning approach," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 3054–3063.
- [9] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, 2018, pp. 1–12.
- [10] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takác, "Reinforcement learning for solving the vehicle routing problem," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 9861–9871.
- [11] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, vol. 134, Oct. 2021, Art. no. 105400.
- [12] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min, "POMO: Policy optimization with multiple optima for reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 21188–21198.
- [13] Y. Wu, W. Song, Z. Cao, J. Zhang, and A. Lim, "Learning improvement heuristics for solving routing problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 5057–5069, Sep. 2022.
- [14] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Mar. 2020.
- [15] C. A. Yano et al., "Vehicle routing at quality stores," *Interfaces*, vol. 17, no. 2, pp. 52–63, 1987.
- [16] P. Toth and D. Vigo, "An exact algorithm for the vehicle routing problem with backhauls," *Transp. Sci.*, vol. 31, no. 4, pp. 372–385, Nov. 1997.
- [17] A. Mingozzi, S. Giorgi, and R. Baldacci, "An exact method for the vehicle routing problem with backhauls," *Transp. Sci.*, vol. 33, no. 3, pp. 315–329, Aug. 1999.
- [18] I. Deif and L. Bodin, "Extension of the Clarke and wright algorithm for solving the vehicle routing problem with backhauling," in *Proc. Babson Conf. Softw. Uses Transp. Logistics Manag.*, Babson Park, MA, USA, 1984, pp. 75–96.
- [19] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Oper. Res.*, vol. 12, no. 4, pp. 568–581, Aug. 1964.
- [20] M. Goetschalckx and C. Jacobs-Blecha, "The vehicle routing problem with backhauls," *Eur. J. Oper. Res.*, vol. 42, no. 1, pp. 39–51, Sep. 1989.
- [21] C. Duhamel, J.-Y. Potvin, and J.-M. Rousseau, "A Tabu search heuristic for the vehicle routing problem with backhauls and time windows," *Transp. Sci.*, vol. 31, no. 1, pp. 49–59, Feb. 1997.
- [22] P. Toth and D. Vigo, "A heuristic algorithm for the symmetric and asymmetric vehicle routing problems with backhauls," *Eur. J. Oper. Res.*, vol. 113, no. 3, pp. 528–543, Mar. 1999.
- [23] P. Toth and D. Vigo, *The Vehicle Routing Problem*. Philadelphia, PA, USA: SIAM, 2002.
- [24] K. Helsgaun, *An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems*. Roskilde, Denmark: Roskilde Univ., 2017, pp. 24–50.
- [25] B. Golden, E. Baker, J. Alfaro, and J. Schaffer, "The vehicle routing problem with backhauling: Two approaches," in *Proc. 21st Annu. Meeting (SE TMS)*, Myrtle Beach, SC, USA, 1985, pp. 90–92.
- [26] H. Hernández-Pérez and J.-J. Salazar-González, "A branch-and-cut algorithm for a traveling salesman problem with pickup and delivery," *Discrete Appl. Math.*, vol. 145, no. 1, pp. 126–139, 2004.
- [27] S. Gélinas, M. Desrochers, J. Desrosiers, and M. M. Solomon, "A new branching strategy for time constrained routing problems with application to backhauling," *Ann. Oper. Res.*, vol. 61, no. 1, pp. 91–109, 1995.
- [28] D. Casco et al., "Vehicle routing with backhauls: Models, algorithms and case studies," *Vehicle Routing, Methods Stud.*, no. 16, pp. 127–147, 1988.
- [29] S. Salhi and G. Nagy, "A cluster insertion heuristic for single and multiple depot vehicle routing problems with backhauling," *J. Oper. Res. Soc.*, vol. 50, no. 10, pp. 1034–1042, 1999.
- [30] K. Ganesh and T. Narendran, "CLOVES: A cluster-and-search heuristic to solve the vehicle routing problem with delivery and pick-up," *Eur. J. Oper. Res.*, vol. 178, no. 3, pp. 699–717, 2007.

- [31] C. Paraphantakul, E. Miller-Hooks, and S. Opananon, "Scheduling deliveries with backhauls in Thailand's cement industry," *Transp. Res. Rec.*, vol. 2269, no. 1, pp. 73–82, 2012.
- [32] N. A. Wassan, G. Nagy, and S. Ahmadi, "A heuristic method for the vehicle routing problem with mixed deliveries and pickups," *J. Scheduling*, vol. 11, pp. 149–161, Apr. 2008.
- [33] M. Avci and S. Topaloglu, "An adaptive local search algorithm for vehicle routing problem with simultaneous and mixed pickups and deliveries," *Comput. Ind. Eng.*, vol. 83, pp. 15–29, May 2015.
- [34] Y. Ma et al., "Learning to iteratively solve routing problems with dual-aspect collaborative transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 11096–11107.
- [35] J. Li et al., "Learning feature embedding refiner for solving vehicle routing problems," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–13, 2023.
- [36] A. I. Garmendia, J. Ceberio, and A. Mendiburu, "Neural improvement heuristics for graph combinatorial optimization problems," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–13, 2023.
- [37] Z. Zhang, Z. Wu, H. Zhang, and J. Wang, "Meta-learning-based deep reinforcement learning for multiobjective optimization problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7978–7991, Oct. 2023.
- [38] Y. Shao et al., "Multi-objective neural evolutionary algorithm for combinatorial optimization problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 2133–2143, Apr. 2023.
- [39] H. Ghaziri and I. H. Osman, "Self-organizing feature maps for the vehicle routing problem with backhauls," *J. Scheduling*, vol. 9, no. 2, pp. 97–114, Apr. 2006.
- [40] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Reinforcement Learn.*, vol. 8, pp. 5–32, May 1992.
- [41] Y. Keneshloo, T. Shi, N. Ramakrishnan, and C. K. Reddy, "Deep reinforcement learning for sequence-to-sequence models," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 7, pp. 2469–2489, Jul. 2019.
- [42] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [43] D. Ulyanov, A. Vedaldi, and V. Lempitsky, "Instance normalization: The missing ingredient for fast stylization," 2016, *arXiv:1607.08022*.
- [44] E. Queiroga, Y. Frota, R. Sadykov, A. Subramanian, E. Uchoa, and T. Vidal, "On the exact solution of vehicle routing problems with backhauls," *Eur. J. Oper. Res.*, vol. 287, no. 1, pp. 76–89, Nov. 2020.
- [45] K. Halse, "Modeling and solving complex vehicle routing problems," Ph.D. dissertation, IMSOR, Tech. Univ. Denmark, Kongens Lyngby, Denmark, p. 372, 1992.
- [46] I. I. C. O. Studio, *V20.1: User's Manual for Cplex*, IBM Corp, Armonk, NY, USA, 2020.
- [47] D. E. Goldberg and R. Lingle, "Alleles, loci, and the traveling salesman problem," in *Proc. Int. Conf. Genetic Algorithms Their Appl.*, vol. 154, Lawrence Erlbaum Hillsdale, NJ, USA, 1985, pp. 154–159.
- [48] S. Chaharsooghi, F. Momayezi, and N. Ghaffarinasab, "An adaptive large neighborhood search heuristic for solving the reliable multiple allocation hub location problem under hub disruptions," *Int. J. Ind. Eng. Comput.*, vol. 8, no. 2, pp. 191–202, 2017.
- [49] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New benchmark instances for the capacitated vehicle routing problem," *Eur. J. Oper. Res.*, vol. 257, no. 3, pp. 845–858, Mar. 2017.
- [50] C. Bai et al., "Variational dynamic for self-supervised exploration in deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 4776–4790, Aug. 2023.
- [51] J. Hao et al., "Exploration in deep reinforcement learning: From single-agent to multiagent domain," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–21, 2023.



Conghui Wang received the B.Eng. degree from the School of Traffic and Transportation Engineering, Central South University, Changsha, China, in 2021, where she is currently pursuing the Ph.D. degree.

Her research interests include vehicle routing and deep reinforcement learning.



Zhiguang Cao received the B.Eng. degree in automation from Guangdong University of Technology, Guangzhou, China, the M.Sc. degree in signal processing from Nanyang Technological University, Singapore, and the Ph.D. degree from the Interdisciplinary Graduate School, Nanyang Technological University.

He was a Research Fellow with the Energy Research Institute, NTU (ERI@N), a Research Assistant Professor with the Department of Industrial Systems Engineering and Management, National University of Singapore, Singapore, and a Scientist with the Agency for Science Technology and Research (A*STAR), Singapore. He joins the School of Computing and Information Systems, Singapore Management University, Singapore, as an Assistant Professor. His research interests focus on learning to optimize (L2Opt).



Yaoxin Wu received the B.Eng. degree in traffic engineering from Wuyi University, Jiangmen, China, in 2015, the M.Eng. degree in control engineering from Guangdong University of Technology, Guangzhou, China, in 2018, and the Ph.D. degree in computer science from Nanyang Technological University, Singapore, in 2023.

He was a Research Associate with the Singtel Cognitive and Artificial Intelligence Laboratory for Enterprises (SCALE@NTU). He joins the Department of Information Systems, Faculty of Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, as an Assistant Professor. His research interests include combinatorial optimization, integer programming, and deep learning.



Long Teng (Member, IEEE) received the Ph.D. degree in control engineering from Nanyang Technological University, Singapore, in 2018.

From August 2017 to September 2018, he worked with Singapore Technology Engineering (Electronics, Satellite Systems), Singapore, and a start-up for rehabilitation robotics, Shenzhen, China. From October 2018 to October 2020, he was a Post-Doctoral Researcher with the Department of Materials and Production, Aalborg University, Aalborg, Denmark. He is currently a Research Assistant Professor with the Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University, Hong Kong. His current research interests include assistive robots, bioinspired robots, control system theory and application to robotics, and machine learning with applications to robotics.



Guohua Wu (Senior Member, IEEE) received the B.S. degree in information systems and the Ph.D. degree in operations research from the National University of Defense Technology, Changsha, China, in 2008 and 2014, respectively.

From 2012 to 2014, he was a Visiting Ph.D. Student with the University of Alberta, Edmonton, ON, Canada. He is currently a Professor with the School of Traffic and Transportation Engineering, Central South University, Changsha. He has authored more than 100 referred papers, including those published in IEEE TRANSACTIONS ON CYBERNETICS (TCYB), IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS, PART A: SYSTEMS AND HUMANS (TSMCA), and IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION (TEVC). His current research interests include planning and scheduling, computational intelligence, and machine learning.

Dr. Wu serves as an Associate Editor for *Information Sciences*, an Associate Editor for *Swarm and Evolutionary Computation Journal*, an Editorial Board Member for *International Journal of Bio-Inspired Computation*, and a Guest Editor of several journals.